دليل التصعيد من الصلاحيات الأفقية إلى الرأسية (Master Cheatsheet)

ربط ثغرات الوصول الجانبي (IDOR) لتحقيق اختراق النطاق الإداري والسيطرة الكاملة.

OWASP A01 + A05

1 المفهوم وبنية الانتقال (Transition Architecture)

التصعيد من الأفقي إلى الرأسي (Horizontal to Vertical Escalation): هي سلسلة استغلال تبدأ بحساب مستخدم عادي (User A)، حيث يستغل المهاجم **ثغرة أفقية (IDOR / BOLA)** للوصول إلى بيانات حساب آخر أعلى منه في الهيكل (مثل Moderator أو Manager)، ومن ثم استغلال وظائف إدارية خفية أو صلاحيات مدمجة داخل هذا الحساب لتصعيد الصلاحيات رأساً **(Vertical Escalation / Admin Takeover)** الوصول للتحكم الكامل بالتطبيق.

2 مسارات الانتقال الشائعة ونقاط الدخول

نوع المسار (Vector)	آلية الاستغلال (Mechanism)	الأثر والنتيجة (Impact)
IDOR إلى إعادة تعيين كلمة السر	تعديل `user_id` في طلب إعادة التعيين لتوليد توكين للـ Admin.	الاستيلاء المباشر على حسابات عالية الصلاحية.
تحديث الملف الشراحي والتلاعب بالأدوار	حقن `"role": "admin"` أو `is_admin":"true` أثناء تحديث البيانات.	تصعيد الصلاحيات عبر Mass Assignment.
سرقة الـ Session / Tokens	الوصول لتوكين جلسة شخص آخر أو API Key مسرب في كائن الملف الشخصي.	وراثة جلسة المدير الإدارية فوراً.
نقاط النهاية الإدارية المخفية	اكتشاف مسارات إدارية أو بارامترات تحكم داخل استجابات المستخدم العادي.	الوصول للوحة التحكم دون صلاحيات رسمية.

3 منهجية الهجوم خطوة بخطوة

1. **الاستطلاع وحصر الحسابات:** تحديد أدوار المستخدمين المعرفة بالأنظمة ومعرفة هيكلية المعرفات (سواء متسلسلة أو UUIDs).
2. **الاستغلال الأفقي الأول:** العثور على ثغرة IDOR تسمح بقراءة أو تعديل بيانات كائنات خاصة بمستخدمين ذوي صلاحيات أعلى.
3. **المحور والتصعيد الرأسي (Pivot):** استغلال التلاعب بالبارامترات، حقول الأدوار، أو توكينات الجلسات المسربة لرفع صلاحية الحساب الحالي إلى أدمن.

4 سيناريوهات واقعية وأكواد الاستغلال

أ. حقن الأدوار عبر Mass Assignment

تعديل حزم الـ JSON لتحديث البيانات بهدف التلاعب بالأدوار وتخزينها في قاعدة البيانات مباشرة.

ب. التحويل عبر IDOR إعادة التعيين

استهداف معرف الحساب الإداري في طلبات إرسال رابط إعادة تعيين كلمة السر للوصول للحساب.

طلب API مصاب (التصعيد الرأسي عبر التلاعب بحقل الدور):

```
PUT /api/v1/user/update_profile HTTP/1.1 Host: target-app.com Authorization: Bearer USER_JWT_TOKEN Content-Type: application/json { "user_id": 105, "first_name": "John", "email": "user@example.com", "is_admin": true, "role": "administrator" }
```

5 قواعد البرمجة الآمنة والمعالجة

❌ كود غير آمن (ربط تلقائي غير معالج / Mass Assignment):

```
@PostMapping("/api/user/update") public User updateUser(@RequestBody User updatedUser, HttpSession session) { // غير آمن: ربط جميع الحقول القادمة في JSON بالكيان مباشرة مما يسمح بتغيير الدور return userRepository.save(updatedUser); }
```

✅ كود آمن (فلترة الحقول الصارمة والتحقق من RBAC):

```
@PostMapping("/api/user/update") public ResponseEntity<String> updateUser(@RequestBody UserDto dto, HttpSession session) { User currentUser = (User) session.getAttribute("currentUser"); // منع تعديل حقول الأدوار بشكل صريح من المستخدمين العاديين if (dto.getRole() != null && !currentUser.hasRole("ADMIN")) { throw new AccessDeniedException("محاولة غير مصرح بها لتعديل الصلاحيات."); } return ResponseEntity.ok("تم تحديث الملف الشخصي بنجاح وآمان."); }
```

6 قائمة التحقق والفحص الميداني

- [ ] اختبار ما إذا كانت نقاط تحديث البيانات تقبل بارامترات أدوار إدارية محظورة (مثل `role=admin`).
- [ ] فحص وجود ثغرات IDOR على نقاط إعادة تعيين كلمة السر أو إدارة الملف الشخصي للحسابات العالية الصلاحية.
- [ ] التحقق من تطبيق الخادم لتقييدات الوصول (RBAC) في كافة عمليات التغيير على مستوى قاعدة البيانات.